⚡ DATABRICKS SPARK SQL CHEAT SHEET

Production Reference: DDL, DML, Liquid Clustering, Delta Lake ACID & Medallion Patterns

SPARK SQL | DBR 15.4+ LTS | PHOTON ENGINE | UNITY CATALOG

🏛️ Unity Catalog 3-Level Namespace & DDL

Governance

```sql
-- 3-Level Hierarchy: catalog.schema.table
CREATE CATALOG IF NOT EXISTS prod_lakehouse;
CREATE SCHEMA IF NOT EXISTS prod_lakehouse.silver_finance
  MANAGED LOCATION 'abfss://silver@storage.dfs.core.windows.net/';
USE CATALOG prod_lakehouse; USE SCHEMA silver_finance;

-- Liquid Clustering (Modern replacement for PARTITION BY)
CREATE TABLE transactions (
  txn_id BIGINT GENERATED ALWAYS AS IDENTITY,
  customer_id STRING,
  amount DECIMAL(18,2),
  txn_date DATE
) USING DELTA CLUSTER BY (txn_date, customer_id);

-- Clones: Zero-copy pointer vs physical backup
CREATE TABLE dev_txns SHALLOW CLONE transactions;
CREATE TABLE dr_backup DEEP CLONE transactions;
```

🔍 CODE BREAKDOWN & ENGINE MECHANICS:

- **Liquid Clustering:** `CLUSTER BY (txn_date, customer_id)` groups data dynamically without rigid Hive directory trees, completely eliminating file fragmentation and partition skew.
- **Identity Sequences:** `BIGINT GENERATED ALWAYS AS IDENTITY` guarantees gapless monotonically increasing primary keys coordinated by the Delta ACID commit log.
- **Managed Storage:** `MANAGED LOCATION` grants Unity Catalog full lifecycle control over underlying ADLS Gen2 blob files; table drop purges storage automatically.
- **Zero-Copy Clone:** `SHALLOW CLONE` creates an instantaneous metadata-only pointer to Parquet data for dev/QA with 0 storage duplication; `DEEP CLONE` copies all physical bytes for DR.

💡 **Architecture Rule:** Liquid clustering replaces Z-Order and static partitions. Cluster keys can be evolved at runtime without costly data restructurings.

🔄 Delta Lake ACID Upsert (MERGE INTO)

DML / Medallion

```sql
MERGE INTO silver.customers AS target
USING bronze.stg_customers AS source
ON target.customer_id = source.customer_id
WHEN MATCHED AND source.is_deleted = true THEN
  DELETE
WHEN MATCHED AND target.hash_diff ≠ source.hash_diff THEN
  UPDATE SET
    target.email = source.email,
    target.status = source.status,
    target.updated_at = current_timestamp()
WHEN NOT MATCHED THEN
  INSERT (customer_id, email, status, created_at, updated_at)
  VALUES (source.customer_id, source.email, source.status,
          current_timestamp(), current_timestamp())
WHEN NOT MATCHED BY SOURCE AND target.status = 'ACTIVE' THEN
  UPDATE SET target.status = 'INACTIVE';
```

🔍 CODE BREAKDOWN & ENGINE MECHANICS:

- **Match Predicate:** `ON target.customer_id = source.customer_id` uses Delta min/max file statistics to prune non-matching Parquet files prior to shuffle join.
- **Delta Change Detection:** `target.hash_diff ≠ source.hash_diff` prevents rewriting unchanged records, minimizing write amplification and storage I/O.
- **Soft Deletes:** `WHEN NOT MATCHED BY SOURCE` handles orphan target records in a single ACID commit, deprecating inactive rows without separate sweeps.
- **ACID Commit:** Writes new Parquet files with updated records and atomically logs file removals in the Delta transaction log `_delta_log/`.

⚡ **Performance:** Always cluster or Z-Order target tables by the MERGE join key to activate Dynamic File Pruning (DFP).

🛠️ Compaction, Z-Order & Vacuum

FinOps / Performance

```sql
-- Compact small files into ~1GB Parquet target files
OPTIMIZE silver.transactions;

-- Multidimensional Space-Filling Z-Order (Legacy/Static tables)
OPTIMIZE silver.transactions ZORDER BY (customer_id, txn_date);

-- Purge unreferenced historical parquet files older than 7 days
VACUUM silver.transactions RETAIN 168 HOURS;

-- Collect CBO statistics for adaptive join planning
ANALYZE TABLE silver.transactions COMPUTE STATISTICS FOR ALL COLUMNS;
```

🔍 CODE BREAKDOWN & ENGINE MECHANICS:

- **OPTIMIZE:** Bin-packs fragmented small files generated by streaming or concurrent writes into uniform ~1GB files, slashing cloud storage API read costs.
- **Z-ORDER BY:** Interleaves high-cardinality column bits using Hilbert curves, enabling massive file skipping during query predicate evaluation.
- **VACUUM RETAIN 168 HOURS:** Deletes orphaned physical Parquet files committed before the retention window, reclaiming storage without breaking concurrent queries.
- **ANALYZE TABLE:** Computes column cardinalities, null counts, and size metrics for Catalyst Cost-Based Optimization (CBO) and join reordering.

⚠️ **Vacuum Caution:** Never set retention < 168 hours in production. Vacuuming 0 hours breaks concurrent active readers and invalidates time travel.

⏳ Delta Time Travel, Audit & Instant Rollback

Resilience

```sql
-- Query historical snapshot by version integer
SELECT * FROM silver.transactions VERSION AS OF 14;

-- Query snapshot by UTC timestamp string
SELECT * FROM silver.transactions
TIMESTAMP AS OF '2026-10-15 08:30:00';

-- Instant point-in-time recovery (Disaster Rollback)
RESTORE TABLE silver.transactions TO VERSION AS OF 13;

-- Inspect full commit history & user audit trail
DESCRIBE HISTORY silver.transactions LIMIT 10;
```

🔍 CODE BREAKDOWN & ENGINE MECHANICS:

- **VERSION / TIMESTAMP AS OF:** Instructs the Delta reader to evaluate transaction log commit entries backward to the exact transaction ID requested.
- **RESTORE TABLE:** Executes instantaneous metadata rollback in under 1 second by creating a new forward commit pointing to prior file manifests.
- **DESCRIBE HISTORY:** Exposes granular operational telemetry including user identities, cluster IDs, job triggers, isolation levels, and write metrics.
- **Storage Safety:** Historical snapshots read preserved older Parquet files directly; no physical backups need to be unpacked or downloaded.

🔒 **Auditability:** Delta time travel is guaranteed up to the VACUUM retention threshold. Use for reproducibility in regulatory reporting.

🔗 Chained CTEs, Window Slicing & Set Operations

Core SQL

```sql
WITH raw_txns AS (
  SELECT customer_id, region, amount,
         DENSE_RANK() OVER(PARTITION BY region ORDER BY amount DESC) AS rk
  FROM silver.transactions WHERE txn_date ≥ current_date() - 30
),
top_spenders AS (
  SELECT customer_id, region, amount FROM raw_txns WHERE rk ≤ 10
)
SELECT * FROM top_spenders
UNION ALL
SELECT customer_id, region, amount FROM silver.manual_overrides
EXCEPT DISTINCT
SELECT customer_id, region, amount FROM silver.fraud_blacklist;
```

🔍 CODE BREAKDOWN & ENGINE MECHANICS:

- **Chained CTEs:** Modularizes complex analytical subqueries; Catalyst inlines CTE expressions into an unified execution plan without materialization.
- **DENSE_RANK():** Assigns consecutive rank integers across partition boundaries without gaps, ideal for top-N ranking per region.
- **UNION ALL:** Appends datasets without triggering an expensive distributed shuffle deduplication stage.
- **EXCEPT DISTINCT:** Computes set difference by executing a left anti-join under the hood, purging fraudulent IDs efficiently.

💡 **Catalyst Optimization:** Predicates applied after CTE definitions (e.g. `rk <= 10`) are automatically pushed down to limit window processing.

🥇 Medallion SQL & QUALIFY Deduplication

Architecture

```sql
-- Silver Cleanse: QUALIFY filter eliminates nested subqueries!
CREATE OR REPLACE TABLE silver.orders_cleansed AS
SELECT
  order_id, customer_id, amount,
  CAST(order_date AS DATE) AS order_date,
  _metadata.file_modification_time AS ingested_at
FROM bronze.orders_raw
WHERE amount > 0 AND order_id IS NOT NULL
QUALIFY ROW_NUMBER() OVER(
  PARTITION BY order_id ORDER BY ingested_at DESC
) = 1;

-- Gold Aggregation: Pre-computed dimension metrics
CREATE OR REPLACE TABLE gold.customer_metrics AS
SELECT customer_id, COUNT(*) AS orders, SUM(amount) AS ltv
FROM silver.orders_cleansed GROUP BY customer_id;
```

🔍 CODE BREAKDOWN & ENGINE MECHANICS:

- **QUALIFY Clause:** Filters windowed calculations directly in the projection, replacing convoluted `SELECT * FROM (SELECT ..., ROW_NUMBER() ...) WHERE rn=1` subqueries.
- **Metadata Columns:** `_metadata.file_modification_time` retrieves file write timestamps directly from cloud storage headers without schema overhead.
- **Idempotent Deduplication:** Partitions by business key and orders by timestamp, ensuring late-arriving updates overwrite older state deterministically.
- **Gold Aggregation:** Pre-aggregates high-cardinality metrics for low-latency BI dashboards and Power BI Direct Lake consumption.

⚡ **Engine Performance:** QUALIFY enables Catalyst to execute top-1 window pruning during physical execution, reducing shuffle data transfer by up to 90%.

⚡ DATABRICKS SPARK SQL CHEAT SHEET

Analytical SQL, Complex Types, Higher-Order Functions, Pivot & Engine Tuning

ADVANCED SQL | ANALYTICAL FUNCTIONS | ROW FILTERS / MASKS

📊 Window Functions & Analytical Framing

Analytics

```sql
SELECT customer_id, txn_date, amount,
  -- Ranking: Row_Number (unique), Dense_Rank (no gaps)
  ROW_NUMBER() OVER(w_acct ORDER BY txn_date DESC) AS rn,
  DENSE_RANK() OVER(w_acct ORDER BY amount DESC) AS rk,
  -- Offsets: Prior & Next transaction amount
  LAG(amount, 1, 0.0) OVER(w_acct ORDER BY txn_date) AS prev_amt,
  LEAD(amount, 1, 0.0) OVER(w_acct ORDER BY txn_date) AS next_amt,
  -- Moving 7-Day Rolling Average (Physical rows)
  AVG(amount) OVER(w_acct ORDER BY txn_date
    ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS roll_7d_avg,
  -- Cumulative Year-to-Date Sum (Range)
  SUM(amount) OVER(w_acct ORDER BY txn_date
    RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS ytd_sum
FROM silver.transactions
WINDOW w_acct AS (PARTITION BY customer_id);
```

🔍 CODE BREAKDOWN & ENGINE MECHANICS:

- **Named Window:** `WINDOW w_acct AS (...)` defines reusable window specifications across multiple expressions, eliminating redundant partition declarations.
- **LAG / LEAD:** Offsets row values in constant time without self-joins; default fallback value handles edge null conditions gracefully.
- **ROWS vs RANGE:** `ROWS BETWEEN` operates on physical row offsets; `RANGE BETWEEN` operates on logical value boundaries (e.g. timestamps/dates).
- **Execution Plan:** Windowing routes through a single `WindowExec` physical operator; sorting occurs locally within partition boundaries.

⚡ **Pro Tip:** If source data is clustered/sorted by the window partition key, Catalyst executes windowing without any network shuffle!

🧩 Complex Types: Structs, Arrays & Maps

Semi-Structured

```sql
-- Struct navigation, Array slicing, Map lookup
SELECT
  payload.user.id AS user_id,             -- struct field
  payload.tags[0] AS primary_tag,          -- array 0-index
  payload.attributes['tier'] AS tier,      -- map lookup
  -- Parse raw JSON string into typed Struct
  from_json(raw_payload, 'id BIGINT, name STRING, items ARRAY<STRING>') AS parsed,
  -- Convert Struct into JSON string
  to_json(named_struct('ts', current_timestamp(), 'code', 200)) AS json_str
FROM bronze.events;

-- Explode Array into relational rows
SELECT order_id, item.sku, item.qty, item.unit_price
FROM silver.orders
LATERAL VIEW EXPLODE(line_items) lv AS item;
```

🔍 CODE BREAKDOWN & ENGINE MECHANICS:

- **Struct & Map Navigation:** Dot syntax (`payload.user.id`) and bracket notation (`attributes['tier']`) access nested schema attributes with zero JSON parsing overhead.
- **from_json():** Parses stringified JSON into strongly-typed Catalyst native representations in C++ Photon; enables column pruning on individual fields.
- **named_struct() & to_json():** Dynamically generates composite structs and encodes them to compressed JSON payloads for egress streaming.
- **LATERAL VIEW EXPLODE:** Unrolls multi-valued array elements into multiple relational rows; use `EXPLODE_OUTER` to retain rows with null or empty arrays.

💡 **Storage Recommendation:** Store semi-structured payloads as native Parquet STRUCTs to achieve column pruning and 10x faster query scans.

⚡ Higher-Order Functions (In-Engine Array Ops)

Photon Vectorized

```sql
SELECT order_id,
  -- TRANSFORM: Map elements without exploding!
  TRANSFORM(line_items, x →
    named_struct('sku', x.sku, 'gross', ROUND(x.price * 1.20, 2))
  ) AS taxed_items,

  -- FILTER: Retain only elements matching condition
  FILTER(line_items, x → x.price > 100.0) AS luxury_items,

  -- EXISTS: Returns true if any element satisfies predicate
  EXISTS(line_items, x → x.sku LIKE 'PROMO-%') AS has_promo,

  -- AGGREGATE: Reduce array to scalar sum
  AGGREGATE(line_items, 0.0, (acc, x) → acc + x.price,
            acc → ROUND(acc, 2)) AS total_order_sum
FROM silver.orders;
```

🔍 CODE BREAKDOWN & ENGINE MECHANICS:

- **TRANSFORM(arr, x → ...):** Applies inline lambda transformations directly to elements inside nested arrays without exploding to intermediate rows.
- **FILTER(arr, x → ...):** Prunes array elements matching the boolean condition entirely in-memory within CPU cache lines.
- **EXISTS(arr, x → ...):** Performs fast short-circuit boolean evaluation, terminating the scan as soon as the first matching element is detected.
- **AGGREGATE(arr, zero, (acc, x) → ..., finish):** Folds array elements into a scalar summary value, operating identically to functional fold/reduce.

🚀 **Vectorized Speed:** Higher-Order functions run in CPU registers inside Tungsten/Photon C++, avoiding Java object allocations and row explosion shuffles.

🔒 Unity Catalog Row Filters & Column Masks

Security

```sql
-- 1. Column Masking (Mask card PAN unless in finance)
CREATE OR REPLACE FUNCTION mask_card_pan(pan STRING)
RETURN CASE
  WHEN is_account_group_member('finance_compliance') THEN pan
  ELSE CONCAT('****-****-****-', RIGHT(pan, 4))
END;

ALTER TABLE silver.payments ALTER COLUMN card_number SET MASK mask_card_pan;

-- 2. Row Filtering (Multi-Tenant isolation)
CREATE OR REPLACE FUNCTION filter_regional_tenants(region STRING)
RETURN is_account_group_member('global_admin') OR region = current_user();

ALTER TABLE silver.payments SET ROW FILTER filter_regional_tenants ON (region);
```

🔍 CODE BREAKDOWN & ENGINE MECHANICS:

- **SET MASK:** Enforces deterministic column-level data redaction in Unity Catalog; non-privileged roles view only masked suffixes across all query tools.
- **SET ROW FILTER:** Injects an invisible dynamic SQL predicate into every downstream query plan, guaranteeing multi-tenant data segregation.
- **is_account_group_member():** Checks SCIM/Entra ID directory group memberships in real time during query parsing.
- **Zero-Copy Security:** Security rules execute at the engine level without creating duplicate physical views or filtered tables.

🔒 **Universal Governance:** Row filters and masks apply uniformly across SQL, PySpark, Power BI Direct Lake, and Databricks SQL Warehouses.

🔄 Dynamic Pivot, Unpivot & Grouping Sets

Dimensional Modeling

```sql
-- Dynamic Pivot (Row categories to columnar metrics)
SELECT * FROM (
  SELECT region, product_category, revenue FROM silver.sales
) PIVOT (
  SUM(revenue) FOR product_category IN (
    'Hardware' AS hw, 'Cloud' AS cloud, 'Services' AS svc
  )
);

-- Unpivot (Columnar metrics back to normalized rows)
SELECT region, category, revenue FROM silver.pivoted_sales
UNPIVOT (revenue FOR category IN (hw, cloud, svc));

-- Multi-Dimensional Subtotals with CUBE
SELECT region, product, SUM(revenue) FROM silver.sales
GROUP BY CUBE(region, product);
```

🔍 CODE BREAKDOWN & ENGINE MECHANICS:

- **PIVOT:** Aggregates categorical dimension keys into horizontal columns in a single scan without chained conditional statements.
- **UNPIVOT:** Normalizes denormalized spreadsheet-style columns back into transactional rows suitable for dimensional modeling.
- **GROUP BY CUBE:** Computes all 2^N grouping permutations in a single pass: `(region, product)` , `(region)` , `(product)` , and Grand Total `()` .
- **Performance Note:** Explicitly supplying the `IN (...)` list avoids an expensive pre-aggregation scan to discover unique keys.

💡 **CUBE vs ROLLUP:** CUBE generates all combinatorial cross-product subtotals; ROLLUP generates strictly hierarchical rollups.

⚡ Spark SQL Performance Hints & Config Knobs

Tuning & FinOps

```sql
-- Join & Skew Optimization Hints
SELECT /*+ BROADCAST(dim), SKEW('fact', 'cust_id') */
  fact.order_id, fact.amount, dim.category_name
FROM silver.orders fact
JOIN silver.categories dim ON fact.cat_id = dim.cat_id;

-- Production Performance Session Knobs
SET spark.sql.shuffle.partitions = auto;
SET spark.sql.adaptive.enabled = true;
SET spark.sql.files.maxPartitionBytes = 134217728; -- 128 MB
SET spark.databricks.delta.optimizeWrite.enabled = true;
SET spark.databricks.delta.autoCompact.enabled = true;
```

🔍 CODE BREAKDOWN & ENGINE MECHANICS:

- **BROADCAST Hint:** Replicates lookup tables (<100MB) to all executors, converting expensive SortMergeJoin to shuffle-free BroadcastHashJoin.
- **SKEW Hint:** Forces Adaptive Query Execution (AQE) to detect and split oversized join partitions on the specified key into smaller sub-tasks.
- **maxPartitionBytes (128MB):** Sets target uncompressed split size when reading Parquet from cloud storage, controlling input task counts.
- **optimizeWrite & autoCompact:** Dynamically coalesces partitions during commits to prevent the small-file problem in high-frequency pipelines.

⚡ **Engine Auto-Tuning:** In DBR 15.4+, `spark.sql.shuffle.partitions = auto` lets AQE dynamically tune shuffle partitions at runtime.

⚡ DATABRICKS PYSPARK CHEAT SHEET

Production Reference: DataFrame API, Strict Schemas, Joins, Windowing & Performance

PYSPARK | DBR 15.4+ LTS | DATAFRAME API | DELTA LAKE

🚀 SparkSession Initialization & DBR Sizing

Configuration

```
from pyspark.sql import SparkSession
import pyspark.sql.functions as F
import pyspark.sql.types as T
from pyspark.sql.window import Window as W

spark = (SparkSession.builder
    .appName("Databricks-Production-Pipeline")
    .config("spark.sql.extensions", "io.delta.sql.DeltaSparkSessionExtension")
    .config("spark.sql.catalog.spark_catalog", "org.apache.spark.sql.delta.catalog.DeltaCatalog")
    .config("spark.sql.execution.arrow.pyspark.enabled", "true")
    .config("spark.sql.adaptive.enabled", "true")
    .getOrCreate())
```

🔍 CODE BREAKDOWN & ENGINE MECHANICS:

- **Delta Extensions:** `io.delta.sql.DeltaSparkSessionExtension` registers Delta ACID query parsing and catalog metadata handlers.
- **Catalog Delegation:** `DeltaCatalog` integrates local and Unity Catalog 3-level namespaces for ACID transactions.
- **PyArrow Optimization:** `arrow.pyspark.enabled` enables zero-copy vectorized serialization between JVM and Python worker processes.
- **AQE Enabled:** Activates Adaptive Query Execution for dynamic runtime join conversions and automatic partition coalescing.

⚡ **Serverless Note:** On Databricks Serverless compute, the SparkSession is pre-initialized and instantly auto-tuned for sub-second scaling.

📥 DataFrame Projections, Transforms & Filters

DataFrame API

```
df_curated = (df_raw
    # Filter with pushdown predicate
    .filter((F.col("status") == "ACTIVE") & (F.col("amount") > 0))
    # Multi-column projection and derivation
    .withColumn("tax_amt", F.round(F.col("amount") * 0.18, 2))
    .withColumn("customer_tier",
        F.when(F.col("amount") >= 10000, "PLATINUM")
         .when(F.col("amount") >= 2500, "GOLD")
         .otherwise("STANDARD"))
    # High-performance projection replacing select() + withColumn()
    .selectExpr("customer_id", "amount", "tax_amt", "customer_tier",
                "current_timestamp() as ingested_at")
)
```

🔍 CODE BREAKDOWN & ENGINE MECHANICS:

- **Filter Pushdown:** `.filter(...)` generates a Catalyst Filter operator pushed down into Parquet footer metadata to skip non-matching row groups.
- **F.when().otherwise():** Compiles directly into native JVM conditional bytecode; avoids python UDF context switching.
- **selectExpr():** Evaluates mixed column projections and SQL expressions in a single projection pass without intermediate plans.
- **withColumns() in Spark 3.3+:** Pass a dictionary to add multiple derived columns simultaneously without rebuilding schema trees.

💡 **Optimization:** Chain transformations lazily; Catalyst combines consecutive filter and project operators into a single pipelined stage.

📐 Strict Schema Definition (StructType & StructField)

Data Quality

```
# Explicit schema avoids expensive inferSchema full-scans
txn_schema = T.StructType([
    T.StructField("txn_id", T.LongType(), nullable=False),
    T.StructField("cust_id", T.StringType(), nullable=False),
    T.StructField("amount", T.DecimalType(18, 2), nullable=False),
    T.StructField("attributes", T.MapType(T.StringType(), T.StringType())),
    T.StructField("items", T.ArrayType(T.StringType())),
    T.StructField("event_time", T.TimestampType())
])

# Read with enforced schema and corrupt record safety
df = (spark.read
      .schema(txn_schema)
      .option("mode", "PERMISSIVE")
      .option("columnNameOfCorruptRecord", "_corrupt")
      .json("abfss://landing@adls.dfs.core.windows.net/txns/"))
```

🔍 CODE BREAKDOWN & ENGINE MECHANICS:

- **Explicit Schema:** Eliminates multi-pass storage scans required by `inferSchema=True`, reducing multi-terabyte ingestion time by 80%.
- **Nullable Guarantees:** Setting `nullable=False` enables Catalyst to optimize join plans and bypass redundant null-check branches.
- **PERMISSIVE Mode:** Captures malformed input JSON rows safely into a quarantine column without crashing distributed streaming tasks.
- **Nested Types:** `MapType` and `ArrayType` preserve schema structure while enabling downstream element-level indexing.

⚡ **FinOps Sizing:** Enforcing schemas on read eliminates full file body scans across cloud storage landing zones.

🧰 pyspark.sql.functions (F) Master Reference

Functions

```
df_transformed = (df
    # String transforms
    .withColumn("full_key", F.concat_ws("_", F.col("region"), F.col("cust_id")))
    # Date formatting & truncation
    .withColumn("month_start", F.date_trunc("MM", F.col("txn_date")))
    # Null coalescing
    .withColumn("clean_phone", F.coalesce(F.col("phone"), F.lit("UNKNOWN")))
    # In-memory array deduplication
    .withColumn("unique_tags", F.array_distinct(F.col("tags")))
    # JSON string extraction
    .withColumn("device_id", F.get_json_object(F.col("raw_meta"), "$.device.id"))
)
```

🔍 CODE BREAKDOWN & ENGINE MECHANICS:

- **JVM Native Execution:** Built-in `pyspark.sql.functions` map directly to Catalyst expressions executed in JVM/C++ Photon without Py4J bridge overhead.
- **concat_ws():** Joins strings using a delimiter and skips null arguments automatically, preventing unwanted null outputs.
- **date_trunc():** Truncates timestamps to the requested calendar boundary, accelerating time-series aggregations.
- **get_json_object():** Extracts scalar JSON path attributes using high-speed JSON path scanning without parsing entire payload schemas.

⚡ **Vectorization Rule:** Never implement custom Python UDFs when equivalent native `F.` functions exist.

🔀 Joins, Broadcast Hints & Skew Salting

Optimization

```
# Broadcast Join: Eliminates shuffle for lookup tables (<100MB)
df_joined = df_fact.join(F.broadcast(df_dim), on="cat_id", how="left")

# Semi & Anti Joins (Replaces expensive IN / NOT IN subqueries)
df_active = df_users.join(df_purchases, on="user_id", how="left_semi")
df_churned = df_users.join(df_purchases, on="user_id", how="left_anti")

# Manual 2-Stage Skew Salting (When AQE cannot balance 100M+ key)
df_skewed = df_fact.withColumn("salt", (F.rand() * 10).cast("int"))
df_dim_exploded = (df_dim
    .withColumn("salt_arr", F.array([F.lit(i) for i in range(10)]))
    .withColumn("salt", F.explode("salt_arr")))
df_salted_join = df_skewed.join(df_dim_exploded, on=["key", "salt"])
```

🔍 CODE BREAKDOWN & ENGINE MECHANICS:

- **Broadcast Hash Join (BHJ):** Replicates the small dimension to all executors, eliminating the expensive SortMergeJoin shuffle exchange.
- **Left Semi Join:** Evaluates existence without duplicating parent rows when child records match multiple times.
- **Left Anti Join:** Returns records from the left dataset having zero matches on the right; highly optimized alternative to `NOT IN`.
- **Synthetic Salting:** Distributes skewed hot keys across 10 virtual partition buckets, eliminating straggler tasks holding up the stage.

⚡ **Threshold Knob:** Configure `spark.sql.autoBroadcastJoinThreshold = 104857600` (100MB) to tune auto-broadcast eligibility.

🪟 Analytical Window Specifications (pyspark.sql.Window)

Analytics

```
# Define Window Specifications
w_dept = W.partitionBy("dept_id").orderBy(F.col("salary").desc())
w_roll_7d = (W.partitionBy("account_id")
              .orderBy(F.col("txn_date").cast("timestamp").cast("long"))
              .rangeBetween(-7 * 86400, 0)) # 7 days in seconds

df_analyzed = (df
    .withColumn("dense_rank", F.dense_rank().over(w_dept))
    .withColumn("prev_salary", F.lag("salary", 1, 0).over(w_dept))
    .withColumn("salary_diff", F.col("salary") - F.col("prev_salary"))
    .withColumn("rolling_7d_total", F.sum("amount").over(w_roll_7d))
)
```

🔍 CODE BREAKDOWN & ENGINE MECHANICS:

- **W.partitionBy().orderBy():** Directs Spark to sort rows within partitioned subsets without sorting the entire distributed dataset.
- **rangeBetween(offset, 0):** Computes rolling aggregate windows across epoch timestamps, properly accounting for days with missing records.
- **F.dense_rank():** Computes ranking without rank number gaps for tied values; ideal for identifying top-earners per department.
- **F.lag(col, offset, default):** Inspects prior record values in a single streaming pass across the partition window.

💡 **Range vs Rows:** `rangeBetween` handles continuous value ranges; `rowsBetween` operates strictly on physical row indices.

⚡ DATABRICKS PYSPARK CHEAT SHEET

Delta API, Auto Loader Streaming, Arrow UDFs, Partitioning, Storage & Microbatching

DELTA LAKE API | AUTO LOADER | VECTORIZED UDFS

DeltaTable Python API & Upsert

Delta API

```python
from delta.tables import DeltaTable

delta_tbl = DeltaTable.forName(spark, "silver.customers")

(delta_tbl.alias("t")
 .merge(df_updates.alias("s"), "t.cust_id = s.cust_id")
 .whenMatchedUpdate(condition="s.hash != t.hash", set={
     "email": "s.email",
     "status": "s.status",
     "updated_at": "current_timestamp()"
 })
 .whenNotMatchedInsert(values={
     "cust_id": "s.cust_id", "email": "s.email",
     "status": "s.status", "created_at": "current_timestamp()"
 })
 .execute())
```

CODE BREAKDOWN & ENGINE MECHANICS:

- **DeltaTable.forName():** Binds programmatic Python handles directly to Unity Catalog registered tables without raw path references.
- **Declarative Merge:** Supports conditional updates and inserts; compiles into an atomic ACID commit updating the Delta transaction log.
- **Hash Comparison:** Skips rewriting unchanged rows, minimizing write amplification on Delta Deletion Vectors and Parquet storage.
- **Schema Evolution:** Pass `.withOption("mergeSchema", "true")` to automatically append new columns arriving from source DataFrames.

Deletion Vectors: In DBR 14.1+, enable Deletion Vectors (`delta.enableDeletionVectors = true`) to execute row updates without file rewrites!

Structured Streaming with Auto Loader (cloudFiles)

Streaming Ingestion

```python
df_stream = (spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "json")
    .option("cloudFiles.schemaLocation", "abfss://chk@adls.dfs.core.windows.net/schem
a")
    .option("cloudFiles.schemaEvolutionMode", "addNewColumns")
    .option("cloudFiles.inferColumnTypes", "true")
    .load("abfss://raw@adls.dfs.core.windows.net/landing/"))

query = (df_stream.writeStream
    .format("delta")
    .outputMode("append")
    .option("checkpointLocation", "abfss://chk@adls.dfs.core.windows.net/chk_stream")
    .trigger(availableNow=True)
    .toTable("bronze.events_stream"))
```

CODE BREAKDOWN & ENGINE MECHANICS:

- **Auto Loader (cloudFiles):** Incrementally discovers and processes new cloud files without expensive file listing calls using cloud notifications.
- **Schema Evolution:** `addNewColumns` safely adapts to evolving JSON payloads without throwing runtime streaming validation errors.
- **Checkpoint Location:** Guarantees exactly-once processing state by persisting commit offsets and metadata in durable cloud storage.
- **trigger(availableNow=True):** Processes all pending backlogged data across microbatches, then automatically terminates compute for FinOps savings.

FinOps Note: `availableNow=True` achieves streaming exactly-once semantics with scheduled batch infrastructure costs.

Vectorized Apache Arrow UDFs (@pandas_udf)

Vectorization

```python
import pandas as pd
from pyspark.sql.functions import pandas_udf

@pandas_udf(T.DoubleType())
def calculate_vat(amount: pd.Series, rate: pd.Series) -> pd.Series:
    # SIMD Vectorized NumPy/Pandas calculation in Python worker
    return (amount * (rate / 100.0)).round(2)

df_with_tax = df.withColumn("vat", calculate_vat(F.col("amount"), F.col("tax_rate")))
```

CODE BREAKDOWN & ENGINE MECHANICS:

- **Apache Arrow Zero-Copy:** Transmits column chunks between JVM and Python workers in Apache Arrow columnar format without row serialization.
- **Vectorized Execution:** Python calculations execute on whole Pandas Series using vectorized C/SIMD loops instead of slow row-by-row iteration.
- **Throughput Speed:** Executes up to **100x faster** than standard Python `@udf`, eliminating Py4J bottleneck latency.
- **Memory Batching:** Tune batch sizes via `spark.sql.execution.arrow.maxRecordsPerBatch` (e.g. 10,000) to balance RAM usage.

Type Contract: Python type hints `(pd.Series) -> pd.Series` determine the vectorization execution model in PySpark 3.0+.

Partitioning Strategy (repartition vs coalesce)

Partitioning

```python
# repartition(N): Full shuffle, balances partitions uniformly
df_shuffled = df.repartition(200, "region")

# repartitionByRange: Sorts data across partitions for range queries
df_ranged = df.repartitionByRange(50, F.col("txn_date").desc())

# coalesce(N): Merges partitions locally WITHOUT full network shuffle!
df_collapsed = df_shuffled.coalesce(10)

# Inspect partition distribution
print("Partition Count:", df_collapsed.rdd.getNumPartitions())
```

CODE BREAKDOWN & ENGINE MECHANICS:

- **repartition():** Executes a full cluster network shuffle; essential when increasing partitions or redistributing skewed datasets.
- **repartitionByRange():** Uses sampling to determine partition split points, producing globally sorted outputs across partition boundaries.
- **coalesce():** Merges adjacent physical partitions within executors without network transfer; only use when decreasing partition count.
- **File Target Sizing:** Adjust partition counts prior to writing Delta tables to maintain 100MB-1GB target Parquet files.

Anti-Pattern: Calling `coalesce(1)` on large datasets collapses upstream execution to a single executor, triggering Driver OOM!

Storage Levels & Memory Persistence

Memory Management

```python
from pyspark import StorageLevel

# Deserialized in RAM (Fastest CPU, Highest RAM)
df.cache() # Shorthand for persist(StorageLevel.MEMORY_AND_DISK_DESER)

# Serialized in RAM with Disk Spill (Production Recommended for Large DFs)
df.persist(StorageLevel.MEMORY_AND_DISK_SER)

# Trigger materialization with an action
record_count = df.count()

# Crucial: Unpersist when downstream processing completes!
df.unpersist(blocking=True)
```

CODE BREAKDOWN & ENGINE MECHANICS:

- **cache() vs persist():** `cache()` defaults to deserialized objects in memory; `persist()` allows customizing serialization and disk spill.
- **MEMORY_AND_DISK_SER:** Stores serialized byte arrays; reduces RAM consumption by ~3x and prevents out-of-memory errors via local disk spill.
- **Lazy Evaluation:** Persisting is registered lazily; data is only populated into cache blocks when the first action (e.g. `.count()`) is invoked.
- **unpersist(blocking=True):** Explicitly frees cluster memory immediately, preventing memory leaks across long-running pipelines.

Off-Heap Option: `StorageLevel.OFF_HEAP` stores data in Tungsten managed memory outside the JVM garbage collector, eliminating GC pauses!

Streaming Microbatch Merges (foreachBatch)

Streaming Sink

```python
def upsert_microbatch(microbatch_df, batch_id):
    # Idempotent write per microbatch
    (DeltaTable.forName(spark, "silver.realtime_orders").alias("t")
     .merge(microbatch_df.alias("s"), "t.order_id = s.order_id")
     .whenMatchedUpdateAll()
     .whenNotMatchedInsertAll()
     .execute())

query = (df_stream.writeStream
    .format("delta")
    .foreachBatch(upsert_microbatch)
    .option("checkpointLocation", "abfss://chk@adls.dfs.core.windows.net/orders_chk")
    .start())
```

CODE BREAKDOWN & ENGINE MECHANICS:

- **foreachBatch:** Bridges Structured Streaming and standard batch APIs, allowing arbitrary Delta MERGE logic per microbatch partition.
- **Idempotent Commits:** Using `batch_id` guarantees exactly-once processing even if a microbatch fails and replays from the checkpoint.
- **Multi-Table Fan-Out:** A single stream can write to multiple Delta tables or external databases simultaneously within the microbatch function.
- **whenMatchedUpdateAll():** Dynamically syncs all columns between source and target without requiring explicit column-by-column mapping.

Streaming Rule: Cache `microbatch_df` inside `foreachBatch` if reading it multiple times to prevent re-evaluating the streaming plan.

SPARK ARCHITECTURE & ENGINE INTERNALS

Cluster Topography, JVM Memory Management, Shuffles, Catalyst Optimizer & Sizing

SPARK CORE | CATALYST & TUNGSTEN | JVM ARCHITECTURE | PRODUCTION SIZING

Cluster Topography & Execution Hierarchy

Architecture

```
# Cluster Submission Architecture & Execution Model
Application
 └── Driver (Coordinates DAGScheduler, TaskScheduler, SparkContext)
      └── Job (Triggered by an Action: .count(), .collect(), .save())
           └── Stage (Boundary determined by Shuffle Exchanges)
                └── Task (Individual computation executed on 1 Partition)
```

CODE BREAKDOWN & ENGINE MECHANICS:

- **Driver:** Orchestrates execution, translates logical DataFrame plans into physical execution stages, and monitors executor health.
- **DAGScheduler:** Evaluates operator graphs, splits stages along wide shuffle boundaries, and optimizes task scheduling.
- **TaskScheduler:** Dispatches individual partition tasks to worker executor slots based on data locality preferences (NODE_LOCAL, PROCESS_LOCAL).
- **Executor:** Dedicated JVM process per worker node executing assigned tasks in parallel threads and caching partition blocks.

Golden Rule: 1 Task = 1 CPU Core = 1 Data Partition. Cluster throughput depends on balancing task counts to total available executor cores.

JVM Unified Memory Management (Heap & Off-Heap)

Memory Pools

```
# Total Executor Memory Breakdown (JVM + Off-Heap)
Total Memory = Executor Heap (spark.executor.memory)
             + Memory Overhead (spark.executor.memoryOverhead)
             + Off-Heap Memory (spark.memory.offHeap.size)

JVM Heap Pools:
├── Reserved Memory: 300 MB (Hardcoded internal Spark engine safety)
├── User Memory: (JVM - 300MB) * (1 - 0.6) = ~40% (UDFs, metadata, hashes)
└── Unified Spark Memory: (JVM - 300MB) * 0.6 = ~60%
     ├── Storage Pool: ~50% (Data caching & broadcast variables)
     └── Execution Pool: ~50% (Shuffles, joins, sorts, aggregations)
```

CODE BREAKDOWN & ENGINE MECHANICS:

- **Dynamic Borrowing:** Execution and Storage pools share a unified boundary; execution can evict cached storage blocks if memory is constrained.
- **Memory Overhead:** Off-heap native memory for Py4J communication, Netty shuffle network buffers, and OS page cache (typically 10% of heap).
- **Off-Heap Tungsten:** `spark.memory.offHeap.enabled = true` stores data outside Java GC in native C-style memory, eliminating GC pauses.
- **OOM Prevention:** If execution memory overflows the unified pool, data spills to local NVMe SSD disks (Spill Memory / Spill Disk).

Tuning Knob: `spark.memory.fraction = 0.6` controls the size of the combined execution/storage pool relative to total JVM heap.

Dependency Graph: Narrow vs Wide Transformations

Execution DAG

```python
# Narrow Transformations (1:1 Partition Mapping - NO Network Shuffle)
df_narrow = (df
    .filter("amount > 100")       # Local row filter
    .select("id", "amount")       # Local projection
    .withColumn("tax", F.col("amount") * 0.18)) # Local arithmetic

# Wide Transformations (M:N Shuffle Exchange - Stage Boundary Created!)
df_wide = (df_narrow
    .groupBy("id").agg(F.sum("amount")) # Hash shuffle across nodes
    .distinct()                         # Deduplication shuffle
    .join(df_other, on="id"))           # SortMerge shuffle
```

CODE BREAKDOWN & ENGINE MECHANICS:

- **Narrow Dependencies:** Partitions are processed independently in parallel within the same executor thread without network transfers (pipelined).
- **Wide Dependencies:** Require data redistribution across nodes based on hash partitioning; forces a stage completion boundary.
- **Shuffle Exchange:** Wide transformations serialize data into shuffle blocks on disk, initiating high network I/O across executors.
- **Inspection:** Execute `df.explain()` to verify stage boundaries; look for `Exchange hashpartitioning` operators.

Architect Principle: Structure transformations so narrow operations filter and prune data prior to invoking wide shuffle operations.

Shuffle Mechanics & Spill Bottlenecks

Engine Bottlenecks

```python
# Shuffle Tuning Knobs
# 1. Shuffle write buffer (reduces disk write syscalls)
spark.conf.set("spark.shuffle.file.buffer", "1m") # Default 32k

# 2. Max size of remote fetch requests in flight
spark.conf.set("spark.reducer.maxSizeInFlight", "96m") # Default 48m

# 3. Dedicated shuffle partition count
spark.conf.set("spark.sql.shuffle.partitions", 400)
```

CODE BREAKDOWN & ENGINE MECHANICS:

- **Shuffle Write:** Map tasks partition records into in-memory append-only buffers, sorting by partition hash before flushing to local disk files.
- **Shuffle Read:** Reduce tasks open parallel Netty HTTP connections to pull their respective partition chunks across the cluster network.
- **Spill (Memory):** Size of uncompressed deserialized records overflowing the execution pool that could not be held in RAM.
- **Spill (Disk):** Size of compressed serialized records written to disk; high spill signals severe memory starvation and disk I/O thrashing.

Diagnostic Alert: When Spark UI shows Spill (Memory) > 0, increase partition counts or scale up executor RAM to eliminate disk thrashing.

Catalyst Query Optimizer & Whole-Stage CodeGen

Catalyst / Tungsten

```
# Inspect Catalyst Execution Pipeline
df.explain(mode="formatted")

# 4 Stages of Catalyst:
1. Unresolved Logical Plan: Verifies SQL syntax & AST
2. Analyzed Logical Plan: Resolves column names against Catalog
3. Optimized Logical Plan: Pushes down predicates & prunes columns
4. Physical Plan: Selects algorithms (HashJoin vs SortMerge)
   └── Whole-Stage CodeGen: Emits Java Bytecode (clean for loops)
```

CODE BREAKDOWN & ENGINE MECHANICS:

- **Predicate Pushdown:** Pushes filter conditions directly into storage file scans, skipping entire row groups before data enters JVM memory.
- **Column Pruning:** Reads only projected columns from columnar Parquet storage, avoiding deserialization of unused wide table fields.
- **Cost-Based Optimizer (CBO):** Evaluates table statistics to choose optimal join orders and select broadcast joins automatically.
- **Whole-Stage CodeGen:** Collapses multiple physical operators into a single Java bytecode loop, eliminating virtual function invocation costs.

Plan Indicator: In `df.explain()`, an asterisk `*(1)` `HashAggregate` confirms that the operator is compiled with Whole-Stage CodeGen!

Production Executor Sizing Formula

Cluster Math

```
# Node Hardware: 10 Nodes, 16 Cores, 64 GB RAM per Node
1. Cores per Node for OS/Daemons = 1 Core (15 Cores left for Spark)
2. Optimal Cores per Executor = 5 Cores (Avoids GC thrashing & HDFS bottleneck)
3. Executors per Node = 15 / 5 = 3 Executors
4. Total Executors = (10 Nodes * 3) - 1 (Driver) = 29 Executors
5. RAM per Node for OS = 1 GB (63 GB left for Spark)
6. RAM per Executor = 63 GB / 3 = 21 GB
7. Memory Overhead = max(384MB, 0.10 * 21GB) = ~2.1 GB
8. Executor Heap = 21GB - 2.1GB = 18.9 GB (Set: --executor-memory 19g)
```

CODE BREAKDOWN & ENGINE MECHANICS:

- **5 Cores Golden Ratio:** Allocating >5 cores per executor leads to severe JVM Garbage Collection pause degradation and storage I/O contention.
- **OS Core Headroom:** Leaving 1 core per VM prevents Kubernetes kubelet and Linux background daemons from starving executor threads.
- **Overhead Calculation:** 10% overhead ensures sufficient memory for off-heap buffers, avoiding YARN/K8s container exit code 137 kills.
- **Driver Reservation:** Deducting 1 executor accounts for the ApplicationMaster/Driver node in cluster deployment mode.

Parallelism Rule: Configure `spark.default.parallelism = total_executor_cores * 2 to 3` (For 145 cores = ~300-450 partitions).

⚙️ SPARK ARCHITECTURE & ENGINE INTERNALS

Photon Native Engine, AQE Dynamic Optimization, Spark UI Triage & Skew Mastery

AQE & SKEW | SPARK UI DEBUGGING | PHOTON C++

🚀 Databricks Photon Native Vectorized Engine

Photon C++

```
# Enable Photon Vectorized Acceleration in Databricks
spark.conf.set("spark.databricks.photon.enabled", "true")

# Vectorized Batch Size & Feature Toggles
spark.conf.set("spark.databricks.photon.columnarExecution", "true")
spark.conf.set("spark.databricks.photon.scan.parquet", "true")
```

🔍 CODE BREAKDOWN & ENGINE MECHANICS:

- **Hand-Optimized C++:** Replaces Java bytecode execution paths entirely with native C++ execution compiled for modern CPU architectures.
- **SIMD Vectorization:** Processes columnar arrays in parallel CPU SIMD registers (Single Instruction, Multiple Data) on every clock cycle.
- **Zero JVM GC Overhead:** Eliminates Java heap allocations for intermediate records, preventing JVM GC pause spikes during heavy aggregations.
- **Accelerated Operations:** Accelerates Parquet scans, hash joins, sorting, higher-order functions, and string parsing by up to **5x**.

⚡ **FinOps ROI:** Photon instances cost slightly more DBUs but deliver 3x-5x throughput, lowering the net total compute cost per pipeline run.

🧠 Adaptive Query Execution (AQE) 3 Superpowers

Adaptive Tuning

```
# Enable Adaptive Query Execution Core & Features
spark.conf.set("spark.sql.adaptive.enabled", "true")

# 1. Dynamic Partition Coalescing (Merges small post-shuffle partitions)
spark.conf.set("spark.sql.adaptive.coalescePartitions.enabled", "true")

# 2. Dynamic Join Switching (SortMergeJoin → BroadcastHashJoin)
spark.conf.set("spark.sql.adaptive.autoBroadcastJoinThreshold", 67108864) # 64MB

# 3. Dynamic Skew Join (Splits oversized partitions automatically)
spark.conf.set("spark.sql.adaptive.skewJoin.enabled", "true")
```

🔍 CODE BREAKDOWN & ENGINE MECHANICS:

- **Runtime Re-Optimization:** AQE inspects concrete stage output statistics at runtime shuffle boundaries to dynamically re-plan downstream stages.
- **Partition Coalescing:** Combines thousands of small post-shuffle partitions into target ~128MB partitions, preventing task scheduling overload.
- **Dynamic Join Switching:** If intermediate filtered tables turn out to be <64MB, AQE converts SortMergeJoin to BroadcastHashJoin on the fly.
- **Automatic Skew Join:** Detects skewed partitions (>5x median size) and splits them into parallel sub-partitions to eliminate stragglers.

💡 **Production Note:** AQE is enabled by default in Databricks Runtime 13+; always verify that skew join is active for heavy ETL pipelines.

🔍 Spark UI Diagnostic & Triage Playbook

Diagnostics

```
# Spark UI Metric Diagnosis Framework
1. Stages Tab & Timeline: Look for straggler tasks (Max >> 75th percentile).
2. Executors Tab: Check GC Time percentage (Alert if GC Time > 10%).
3. Task Metrics: Inspect Shuffle Spill (Memory) and Spill (Disk).
4. SQL / Plan Tab: Verify SortMergeJoin vs BroadcastHashJoin selection.
```

🔍 CODE BREAKDOWN & ENGINE MECHANICS:

- **Task Duration Skew:** If 199 tasks finish in 2 seconds but 1 task takes 10 minutes, you have severe key data skew in the shuffle stage.
- **GC Time > 10%:** Signals JVM heap pressure; reduce executor cores or enable off-heap memory to avoid pause cascades.
- **Shuffle Spill Indicator:** Spill indicates partitions are larger than available execution RAM; increase `spark.sql.shuffle.partitions`.
- **DAG Inspection:** Use the SQL tab DAG to find missing partition pruning filters or unexpected CartesianProduct operators.

⚠️ **Triage Playbook:** Always check the Stages tab first; identify the longest stage, then inspect task duration distribution to find bottlenecks.

⚠️ Data Skew Root Causes & Solutions

Skew Remediation

```
# Strategy: Separate Nulls & Extreme Hot-Keys
df_non_null = df_fact.filter(F.col("cust_id").isNotNull())
df_nulls = df_fact.filter(F.col("cust_id").isNull())

# Join clean data with dimension, then re-attach nulls without shuffle
df_joined = (df_non_null
    .join(df_dim, on="cust_id", how="left")
    .unionByName(df_nulls.selectExpr("* ", "null as dim_col")))
```

🔍 CODE BREAKDOWN & ENGINE MECHANICS:

- **Root Cause:** High-frequency keys (e.g. `NULL` customer IDs, default dummy values like -1) hash into a single partition executor.
- **Null Isolation:** Null values never match in inner/left joins; isolating nulls bypasses the shuffle hash collision entirely.
- **Broadcast Join:** If one side of the join is <8GB on large clusters, broadcasting it completely bypasses the shuffle exchange.
- **Synthetic Salting:** Appending random integer suffixes (0–9) spreads skewed keys across 10 independent executors in parallel.

💡 **Detection:** Run `df.groupBy("key").count().orderBy(F.desc("count")).show(10)` to identify high-cardinality skew keys.

💾 Storage Levels & Checkpoint Decision Matrix

Persistence

```
# persist() keeps DAG lineage for node failure recovery
df_cached = df.persist(StorageLevel.MEMORY_AND_DISK_SER)

# checkpoint() writes to ADLS and TRUNCATES lineage graph!
spark.sparkContext.setCheckpointDir("abfss://chk@adls.dfs.core.windows.net/dag")
df_checkpointed = df.checkpoint(eager=True)
```

🔍 CODE BREAKDOWN & ENGINE MECHANICS:

- **persist():** Retains the full logical DAG lineage; if an executor fails, Spark recomputes only the missing partition from source.
- **checkpoint():** Writes physical data to durable cloud storage and completely **truncates the lineage graph**.
- **StackOverflow Prevention:** Mandatory in iterative machine learning loops (ALS, GraphX) and complex streaming joins to avoid DAG overflow.
- **Storage Level Selection:** Use `MEMORY_AND_DISK_SER` in production to minimize JVM GC pauses while avoiding OOM spills.

🔒 **Architect Rule:** Use `persist()` for reusing intermediate DataFrames in the same job; use `checkpoint()` to break huge recursive lineages.

⚙️ JVM Garbage Collection (G1GC) & Shuffle Math

Tuning Knobs

```
# Production G1GC Flags for Databricks / Spark
-XX:+UseG1GC
-XX:InitiatingHeapOccupancyPercent=35
-XX:G1ReservePercent=15
-XX:MaxGCPauseMillis=200

# Optimal Shuffle Partitions Calculation Formula:
# Rule: Target 100MB - 200MB uncompressed per shuffle partition
Total Shuffle Input Data = 64 GB (= 65,536 MB)
Target Partition Size = 128 MB
Optimal Partitions = 65,536 MB / 128 MB = 512 Partitions
Config: spark.conf.set("spark.sql.shuffle.partitions", 512)
```

🔍 CODE BREAKDOWN & ENGINE MECHANICS:

- **G1GC Collector:** Partitions heap into 2048 regions; prioritizes garbage collection in regions containing mostly dead objects.
- **InitiatingHeapOccupancyPercent (35%):** Starts concurrent marking earlier, preventing stop-the-world Full GC pause cascades.
- **MaxGCPauseMillis (200ms):** Sets soft target pause duration for the JVM collector to balance throughput and latency.
- **Shuffle Partition Math:** Oversized partitions (>500MB) cause OOM spills; undersized partitions (<10MB) introduce scheduling overhead.

⚡ **AQE Advisory Size:** Set `spark.sql.adaptive.advisoryPartitionSizeInBytes = 134217728` (128MB) for automatic post-shuffle partition sizing.

PYTHON IN DATABRICKS CHEAT SHEET

Production Reference: DBUtils, Delta Live Tables (DLT), pandas on Spark, SDK & Magic Commands

PYTHON 3.10+ | DBUTILS | DELTA LIVE TABLES | DATABRICKS SDK

dbutils Master Reference (fs, secrets, notebook)

Databricks Utilities

```python
# 1. File System Operations (Cloud Storage Navigation)
files = dbutils.fs.ls("/Volumes/prod_catalog/raw_landing/")
dbutils.fs.mv("/tmp/source.csv", "/mnt/datalake/destination.csv")

# 2. Secret Management (Azure Key Vault / AWS Secrets Manager)
api_token = dbutils.secrets.get(scope="kv_fin_prod", key="payment_gateway_key")

# 3. Parameterized Widgets (Pass arguments to notebooks)
dbutils.widgets.text("env", "prod", "Environment")
current_env = dbutils.widgets.get("env")

# 4. Notebook Workflow Orchestration & Parameter Passing
result = dbutils.notebook.run("sub_etl", timeout_seconds=1800, arguments={"batch_id": "102"})
dbutils.notebook.exit('{"status": "SUCCESS", "records": 45000}')
```

CODE BREAKDOWN & ENGINE MECHANICS:

- **dbutils.fs:** Interacts directly with ADLS/S3 storage drivers, abstracting distributed cloud credentials and API calls.
- **dbutils.secrets.get():** Redacts sensitive API keys and database credentials automatically from notebook outputs and cluster logs.
- **dbutils.widgets:** Injects runtime parameters into notebook cells from Airflow, Azure Data Factory, or Databricks Workflows.
- **dbutils.notebook.exit():** Serializes structured JSON return payloads back to parent workflow orchestrators.

Security Hardening: Never hardcode secrets in notebooks; always configure Unity Catalog Secret Scopes with Key Vault backing.

Delta Live Tables (DLT) Declarative Pipelines

Declarative ETL

```python
import dlt
from pyspark.sql.functions import col, expr

@dlt.table(
    name="bronze_raw_txns",
    comment="Raw incremental streaming transactions"
)
def bronze_txns():
    return (spark.readStream.format("cloudFiles")
            .option("cloudFiles.format", "json")
            .load("/Volumes/prod/raw/txns"))

@dlt.table(name="silver_clean_txns")
@dlt.expect_or_drop("valid_amount", "amount > 0")
@dlt.expect_or_fail("valid_id", "txn_id IS NOT NULL")
def silver_txns():
    return dlt.read_stream("bronze_raw_txns").filter(col("status") == "SETTLED")
```

CODE BREAKDOWN & ENGINE MECHANICS:

- **@dlt.table:** Declares a managed Delta table; DLT constructs the global DAG dependency graph and manages cluster provisioning.
- **dlt.read_stream():** Binds streaming table dependencies, propagating schema changes and updates through the Medallion architecture.
- **@dlt.expect_or_drop:** Enforces data quality rules; drops invalid rows while recording violation metrics in the pipeline event log.
- **@dlt.expect_or_fail:** Halts pipeline execution immediately if critical schema or business invariants are breached.

Serverless DLT: Serverless DLT pipelines eliminate cluster spin-up time and dynamically allocate compute as streaming queues fluctuate.

Streaming CDC with dlt.apply_changes() (SCD 1 & 2)

Change Data Capture

```python
import dlt

# Declare destination streaming table
dlt.create_streaming_table("silver_dim_customers")

# Turnkey Slowly Changing Dimension (SCD Type 2)
dlt.apply_changes(
    target="silver_dim_customers",
    source="bronze_cdc_customers",
    keys=["customer_id"],
    sequence_by="operation_timestamp",
    apply_as_deletes=col("op_type") == "D",
    except_column_list=["op_type", "ingested_at"],
    stored_as_scd_type=2 # 1 = In-Place Overwrite; 2 = Historical Tracking
)
```

CODE BREAKDOWN & ENGINE MECHANICS:

- **dlt.apply_changes():** Native declarative CDC engine supporting out-of-order Debezium / Fivetran / Oracle GoldenGate events.
- **sequence_by:** Monotonic sequence key or timestamp ensuring out-of-order CDC packets are resolved deterministically.
- **stored_as_scd_type=2:** Automatically maintains validity tracking columns: `__START_AT`, `__END_AT`, and active flags.
- **apply_as_deletes:** Evaluates soft or hard delete operations, setting `__END_AT` without physically corrupting history.

SCD Type 1 vs 2: Use SCD 1 for current-state lookup dimensions; use SCD 2 for point-in-time compliance and financial audits.

Pandas API on Spark (pyspark.pandas as ps)

Distributed Pandas

```python
import pyspark.pandas as ps

# 100% Pandas Syntax executed across distributed Spark Cluster!
psdf = ps.read_parquet("abfss://silver@adls.dfs.core.windows.net/orders")

# High-level analytical calculations with no PySpark boilerplate
psdf["tax"] = psdf["amount"] * 0.18
summary = psdf.groupby("region")["amount"].agg(["mean", "sum", "std"])

# Seamless Two-Way Conversion
spark_df = psdf.to_spark()
ps_df_again = spark_df.to_pandas_on_spark()
```

CODE BREAKDOWN & ENGINE MECHANICS:

- **Distributed Pandas:** Translates standard Pandas method invocations directly into distributed Spark Catalyst query execution plans.
- **No Driver Memory Limit:** Unlike native Pandas, data remains distributed across worker executor memory; processes 100GB+ seamlessly.
- **to_spark() / to_pandas_on_spark():** Zero-copy conversion between PySpark DataFrames and Pandas-on-Spark representations.
- **Index Management:** Uses default distributed indexing; avoid setting sequential range indexes on huge datasets to prevent sorting shuffles.

Anti-Pattern: Avoid calling `.to_pandas()` or `.iloc` on large DataFrames as it pulls all distributed data into driver RAM, risking OOM!

Databricks Connect v2 & Databricks Python SDK

Developer Tooling

```python
# Databricks Connect v2 (Local IDE to Remote Cluster over gRPC)
from databricks.connect import DatabricksSession
spark = DatabricksSession.builder.getOrCreate()
df = spark.read.table("prod_catalog.silver.transactions")
df.filter("amount > 1000").show()

# Databricks Python SDK (Workspace Automation)
from databricks.sdk import WorkspaceClient
w = WorkspaceClient()
run = w.jobs.run_now(job_id=892145)
clusters = w.clusters.list()
```

CODE BREAKDOWN & ENGINE MECHANICS:

- **Databricks Connect v2:** Built on the open-source Spark Connect protocol; local Python code runs thin clients over secure gRPC to remote clusters.
- **Local IDE Debugging:** Set breakpoints in VS Code or PyCharm, step through code interactively, and execute against production cluster data.
- **WorkspaceClient:** Official SDK managing all Databricks REST API endpoints: Jobs, Clusters, Unity Catalog, Warehouses, and Secrets.
- **Unified Authentication:** Automatically resolves credentials from environment variables (`DATABRICKS_HOST`, `DATABRICKS_TOKEN`) or Azure CLI.

Thin Client: Databricks Connect v2 sends unresolved DataFrame plans via gRPC; execution runs entirely on the remote Databricks cluster.

Databricks Magic Commands & Scoping

Notebook Magic

```python
# 1. SQL Magic & Accessing Results in Python
%sql
SELECT customer_id, SUM(amount) AS total FROM silver.orders GROUP BY customer_id

# Access SQL query output in next Python cell:
sql_df = _sqldf.filter("total > 5000")

# 2. Ephemeral Environment Setup
%pip install pydantic==2.8.2 tenacity==8.4.1

# 3. Modular File Execution
%run ./common_utilities_notebook
```

CODE BREAKDOWN & ENGINE MECHANICS:

- **%sql & _sqldf:** Running a `%sql` cell automatically binds the output DataFrame to the `_sqldf` variable in downstream Python cells.
- **%pip:** Installs isolated dependencies onto the cluster driver and worker environments for the duration of the notebook session only.
- **%run:** Executes external shared notebooks in-line, inheriting functions, imports, and variables directly into the current notebook session.
- **%sh & %fs:** Executes bash commands inside the driver container and navigates storage volumes via command-line utilities.

%run vs dbutils.notebook.run: `%run` shares memory & variables in the same session; `dbutils.notebook.run` executes in an isolated ephemeral job context.

🐍 PYTHON IN DATABRICKS CHEAT SHEET

Wheels, Databricks Asset Bundles (DABs), Observability, Resilience, Types & Unit Testing

DABS / CI/CD | PRODUCTION WHEELS | OBSERVABILITY

📦 Wheels & Unity Catalog Volumes

Packaging

```
# In-Notebook ephemeral dependency install
%pip install pydantic==2.8.2 tenacity==8.4.1

# Production Wheel Distribution via Unity Catalog Volumes
# Store built wheel in governed volume:
# /Volumes/prod_catalog/core_libs/my_etl-1.2.0-py3-none-any.whl
%pip install /Volumes/prod_catalog/core_libs/my_etl-1.2.0-py3-none-any.whl

# Import modular reusable package in notebook
from my_etl.transformers import BankingNormalizer
normalizer = BankingNormalizer(currency="EUR")
```

🔍 CODE BREAKDOWN & ENGINE MECHANICS:

- **Unity Catalog Volumes:** Governed storage locations in cloud storage supporting file-level RBAC, audit logging, and binary package versioning.
- **Custom Wheels (.whl):** Package enterprise business logic into modular Python wheels to enforce automated CI/CD unit testing and release tagging.
- **Cluster Libraries:** Attach volume paths to cluster configurations or Job task dependencies to pre-install packages across worker nodes.
- **Zero DBFS Root:** Completely replaces insecure legacy DBFS root paths with enterprise-governed Unity Catalog storage locations.

💡 **Volumes for Libraries:** Replacing legacy DBFS root with Unity Catalog Volumes ensures strict access control and auditing for third-party binary artifacts.

🚀 Databricks Asset Bundles (DABs) CI/CD

DevOps

```
# databricks.yml definition snippet
bundle:
  name: bfsi_settlement_pipeline

targets:
  dev:
    workspace:
      host: https://adb-dev.azuredatabricks.net
  prod:
    workspace:
      host: https://adb-prod.azuredatabricks.net

resources:
  jobs:
    nightly_settlement:
      name: "[prod] Nightly Settlement Job"
      tasks:
        - task_key: run_dlt
          pipeline_task:
            pipeline_id: 489a-bc12-dlt-id
```

🔍 CODE BREAKDOWN & ENGINE MECHANICS:

- **Infrastructure-as-Code (IaC):** Declaratively defines Databricks workflows, notebooks, DLT pipelines, and permissions in source-controlled YAML.
- **Multi-Environment:** `targets: dev / prod` abstracts workspace host URLs and cluster settings, enabling clean promotion across stages.
- **CLI Commands:** `databricks bundle validate` (linting), `bundle deploy -t prod` (deployment), and `bundle run` (execution).
- **GitHub Actions / Azure DevOps:** Integrates directly into CI/CD runners using service principals without manual workspace UI clicking.

⚡ **Modern DevOps:** Databricks Asset Bundles (DABs) are the standard for deploying enterprise data engineering projects in 2025/2026.

📊 Structured Logging & Observability

Observability

```
import json, logging
from datetime import datetime

class StructuredPipelineLogger:
    def __init__(self, pipeline_name: str):
        self.name = pipeline_name

    def log_event(self, event_name: str, records: int, duration_sec: float):
        payload = {
            "timestamp": datetime.utcnow().isoformat(),
            "pipeline": self.name,
            "event": event_name,
            "records_processed": records,
            "duration_sec": duration_sec
        }
        print(json.dumps(payload)) # Ingested into Azure Log Analytics

audit = StructuredPipelineLogger("SEPA_Payment_Processor")
audit.log_event("STAGE_COMPLETED", records=845000, duration_sec=14.2)
```

🔍 CODE BREAKDOWN & ENGINE MECHANICS:

- **Single-Line JSON:** Emitting serialized JSON to standard stdout ensures cloud observability scrapers (Datadog, Splunk, Azure Log Analytics) parse attributes automatically.
- **Metrics Tracking:** Records throughput metrics, processing duration, and stage identifiers for SLI/SLA compliance monitoring.
- **Distributed Driver Scoping:** Instantiated on the driver node to capture macro-level stage completions and pipeline milestones.
- **Alerting Triggers:** Enables automated incident triggers when duration spikes above historical 95th percentile thresholds.

🔒 **Enterprise Standard:** Standardized JSON logging links cluster run IDs and pipeline commits directly into enterprise monitoring dashboards.

🔄 Concurrent Extraction & Exponential Retries

Resilience

```
from concurrent.futures import ThreadPoolExecutor
from tenacity import retry, stop_after_attempt, wait_exponential

@retry(stop=stop_after_attempt(3), wait=wait_exponential(multiplier=1, min=2, max=10))
def fetch_api_page(page_id: int) -> dict:
    # Resilient REST API caller with exponential backoff
    return {"page": page_id, "data": [1, 2, 3]}

# Parallelize lightweight I/O on Driver before Spark processing
with ThreadPoolExecutor(max_workers=8) as executor:
    results = list(executor.map(fetch_api_page, range(1, 50)))

df_api = spark.createDataFrame(results)
```

🔍 CODE BREAKDOWN & ENGINE MECHANICS:

- **Driver Multithreading:** Uses Python `ThreadPoolExecutor` on the driver node to parallelize I/O-bound REST API web requests efficiently.
- **tenacity @retry:** Catches transient HTTP 429 rate-limiting and 503 gateway timeout exceptions, backing off exponentially before retrying.
- **createDataFrame():** Converts accumulated JSON page dictionaries into a distributed PySpark DataFrame across worker executors.
- **Separation of Concerns:** Use Python threads for I/O fetching; use Spark distributed partitions for data transformations and writes.

⚡ **Thread Concurrency:** Use Python `ThreadPoolExecutor` on the driver for I/O-bound REST calls, then parallelize the dataset with `spark.createDataFrame()`.

🛡️ Type Annotations & Schema Contracts

Data Quality

```
from pyspark.sql import DataFrame, Column
from typing import Callable

def apply_curation(
    df: DataFrame,
    primary_key: str,
    business_rule: Callable[[Column], Column]
) -> DataFrame:
    # Pre-flight contract validation
    assert primary_key in df.columns, f"Missing required key: {primary_key}"
    return df.filter(business_rule(df[primary_key]))
```

🔍 CODE BREAKDOWN & ENGINE MECHANICS:

- **PEP 484 Typing:** Explicitly annotates PySpark DataFrames, Columns, and function callables to enable static type analysis with `mypy`.
- **Pre-Flight Assertions:** Validates input schemas before submitting heavy distributed compute jobs, failing in 10ms instead of 2 hours later.
- **Higher-Order Parameterization:** Accepts Column lambda functions (`Callable[[Column], Column]`) for modular rule composition.
- **Documentation Integrity:** Serves as self-documenting code contracts across enterprise data engineering squads.

💡 **Fail Fast:** Pre-flight contract validation prevents pipelines from burning expensive compute hours on downstream schema errors.

🧪 Unit Testing PySpark Pipelines in CI/CD

Testing

```
import pytest
from pyspark.testing.utils import assertDataFrameEqual

def test_filter_active(spark):
    input_data = [(1, "ACTIVE"), (2, "CANCELLED")]
    input_df = spark.createDataFrame(input_data, ["id", "status"])

    expected_data = [(1, "ACTIVE")]
    expected_df = spark.createDataFrame(expected_data, ["id", "status"])

    result_df = input_df.filter("status = 'ACTIVE'")
    # Spark 3.5+ built-in test assertion
    assertDataFrameEqual(result_df, expected_df)
```

🔍 CODE BREAKDOWN & ENGINE MECHANICS:

- **assertDataFrameEqual:** Built into PySpark 3.5+; compares distributed DataFrames, ignoring row ordering and providing detailed schema diffs.
- **Tolerance Support:** Supports `atol` and `rtol` arguments to accommodate floating-point precision differences across environments.
- **Zero External Dependencies:** Eliminates third-party test libraries (e.g. `chispa`), simplifying GitHub Actions / Azure DevOps CI runners.
- **Pytest Fixtures:** Pass a shared session-scoped `spark` fixture to run unit test suites locally in under 5 seconds.

⚡ **CI/CD Speed:** Run PySpark unit tests using local mode (`local[2]`) in CI runners to validate logic prior to deploying Asset Bundles.